UNIVERSITÉ CÔTE D'AZUR | MASTER 2 – SLCD

Big Data and NoSQL

Project 3 – UrbanFlow

An Urban Mobility Analytics Platform

Hajer Akid | Academic year 2026–2027

Groups	Two students. All members must be able to explain any part of the submitted work.
Submission	**Friday, 6 November 2026**
Defence	**Tuesday, 10 November 2026, at 2:00 p.m.**
Deliverables	A **written report** (at most 10 pages, excluding appendices); the **source code**; a **15-minute oral presentation** including a live demonstration, followed by a **10-minute question-and-answer session** with a live challenge.

1. Project scenario

A municipal mobility analysis team wants to understand how recorded taxi activity varies across the city. It needs to identify busy departure areas, measure journeys between neighbourhoods, compare traffic patterns across periods and investigate unusual changes without manually processing millions of individual trip records.

You are commissioned to develop **UrbanFlow**, a working platform through which an analyst can explore historical journeys, investigate flows between urban areas and obtain evidence for the reported patterns. The analyst must be able to change the geographical scope, period and conditions of an investigation without rebuilding the system, and every reported pattern must be traceable to the trip records that support it.

UrbanFlow will first run on a **single local data server**. It will then be deployed using **several data-serving instances** on the same workstation. The two versions must provide the same services and identical results, while allowing their ingestion costs, query behaviour and data distribution to be compared.

2. Data

Use the **New York City Taxi and Limousine Commission (TLC) Yellow Taxi Trip Records**. The monthly Parquet files contain recorded taxi trips, with pickup and drop-off timestamps, origin and destination taxi zones, trip distances, fares and payment information. Combine them with the official taxi-zone lookup table, which provides zone and borough names.

Resource	Official source
Trip records and zone lookup	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
Taxi-zone lookup (CSV)	https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
Field definitions	https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf

Each monthly file contains roughly three million trips. Read the data dictionary before interpreting prices, distances, payment types or codes. In particular:

- timestamps are local New York times without time-zone information, and 2024 includes two daylight-saving transitions;
- a monthly file may contain trips whose pickup date falls outside that month;
- the records have **no trip identifier**; exact duplicate rows are rare, but many trips appear twice, as an original record and a reversal record with negated amounts (same vendor, timestamps and zones);
- some zone codes do not correspond to a precise area (unknown or outside the city), zone names are not unique identifiers, and zones are areas, *not* the routes followed by vehicles.

Completed yellow-taxi trips alone cannot establish unmet passenger demand or citywide travel behaviour; your conclusions must stay within what the data record.

Period, focus area and registration

Use **at least three consecutive complete months from 2024**, including a month with a daylight-saving transition (March or November), in the initial platform, and reserve **the following complete month** as the update batch for service F6. Retain the detailed trip records, not only aggregates. Choose a **focus borough or group of zones** on which the detailed examples of the report will concentrate; the platform itself must still cover the whole city.

Each group registers its months and focus area with the instructor before starting the implementation. **No two groups may register the same combination**; the instructor may ask a group to modify a choice that overlaps too much with another one. The instructor will use the registered choices to prepare the defence challenges (Section 6).

Provenance and quality

Record the retrieval date, the exact list of source files and a checksum of each of them. Report the number of trips per file, the rule used to assign a trip to a period, and the spatial coverage. Define which records remain usable for each analysis and report the effect of filtering.

In an appendix, document **at least five data anomalies actually present in your files** (for example: pickup dates outside the file's month, negative or zero durations, implausible distances or speeds, negative fares, unknown zones, duplicate rows, trips spanning a daylight-saving change), each with its frequency and at least one concrete record. These examples will be checked against the source files.

All services and performance experiments must operate on **genuine TLC trip records**. Do not manufacture trips, duplicate records to increase volume or treat replicated copies as additional journeys.

3. System to develop

Provide the following services through a usable interface (web interface, API or command-line application). Zones, areas (a set of zones or a borough) and time intervals are specified at execution time; results must be computed from the stored data. Pre-aggregated tables or materialised views are allowed if the system maintains them and F6 keeps them up to date; answers prepared in advance for fixed examples, or tables assembled by hand, are not.

F1. Explore activity in a zone. Given a pickup or drop-off zone and a date interval, describe the number and temporal distribution of trips. Compare activity across hours and days of the week; expose the number of records contributing to each result and the effect of data-quality exclusions.

F2. Analyse origin–destination flows. Given an area and a period, identify the main destinations of trips originating there and the most active origin–destination pairs. Compare travel in opposite directions, distinguish consistently frequent links from short-lived peaks, and avoid interpreting a small number of trips as a major flow.

F3. Investigate unusual activity. Detect zones and time windows with markedly higher or lower recorded activity than comparable periods. Define the reference periods, the minimum evidence required and the criteria for an unusual change, and take public holidays into account. Explain whether a reported deviation could be caused by unequal coverage or irregular source records.

F4. Compare journey characteristics. For selected zone pairs and time intervals, study travel duration and distance, including differences between typical journeys and less reliable ones. Durations must remain correct for trips spanning a daylight-saving transition. Compare distinct periods while accounting for outliers, sample size and variation in trip characteristics, and return interpretable statistics rather than a single overall average.

F5. Measure changes in urban flows. Given two comparable periods, identify origin–destination pairs with the largest increases or decreases in their share of recorded journeys. With millions of trips, a classical significance test flags almost every change: assess changes against the normal week-to-week variation of each flow. Distinguish changes in a particular flow from changes in the overall number of recorded trips; provide the supporting counts and explain the ranking criterion.

F6. Update the archive. Import the reserved month without losing previous data or creating duplicates. Since the source has no trip identifier, define the identity of a trip and justify how genuine identical trips are distinguished from a record imported twice. Importing the same file twice must leave the stored data unchanged. The update must work on both deployments, and the services above must reflect the new month.

For each service, specify the accepted inputs and returned outputs and present at least one example drawn from the real files. Any reported result must be traceable: given a result, the system must be able to list or count the underlying trip records with their source file.

4. Implementation and distribution

Part I – Local prototype

Design a representation of the trip records and taxi-zone information, then implement all six services on a single local data server, that is, a database system running as a separate server process queried by the application through a network endpoint (an embedded library such as SQLite or DuckDB qualifies only if it is wrapped in such a server process). Describe the data-management technology chosen, the information stored, the expected access patterns and the mechanisms used to support the required operations efficiently.

Use actual measurements to justify at least one important design decision. Implement and measure **one alternative design** (data model, pre-aggregation, partitioning or indexing) on the same data and on at least two representative requests, and explain why it was retained or rejected. If pre-aggregated data are used, show that they remain consistent with the detailed records after an update. Ingestion must be reproducible.

Part II – Distributed prototype

Build a second deployment with **at least two separate data-serving instances**, running as processes or containers with distinct endpoints and independent storage. A data-serving instance is a database or storage server that stores part of the trip records and answers queries on them;

a processing engine alone (for instance Spark reading Parquet files) does not count as one. Trip records must be placed across these instances rather than merely duplicated in full; the taxi-zone lookup may be replicated on every instance. UrbanFlow must continue to offer a single interface for all six services.

Define and justify a placement strategy in view of the temporal and geographical distribution of the data. Examine how it affects a request centred on a single busy pickup zone, a request spanning several zones and an origin–destination comparison across the city. For each, show which instances are contacted, which computations are performed locally and which intermediate results are exchanged and combined. Verify that the local and distributed versions produce **identical results** for identical trips and parameters, using an automated comparison. Compare results after a canonical ordering, with ties broken deterministically. The comparison must include requests whose time range or set of zones spans several instances.

Implement and evaluate **one alternative distributed configuration**, changing the placement strategy or the number of data-serving instances.

Losing an instance. Stop one data-serving instance and run F1 on a zone and period whose trips are partly stored on that instance, then F5. Show what the user receives. A partial answer must never be presented as complete: either the request fails explicitly, or the response states which data were unavailable. Distinguish what your implementation actually recovers from what a replicated production system could recover. A multi-process deployment on one workstation must not be presented as equivalent to a cluster of independent machines.

5. Experimental investigation

The mobility team expects both frequent, narrowly scoped enquiries and occasional citywide analyses. Activity is also uneven: some zones and hours account for far more recorded trips than others.

E1. **Data volume and loading.** Construct three increasingly large datasets from *complete months of the actual TLC archive*, the largest covering **at least six months**. Measure ingestion time, storage requirements and latency for representative services, and give the number of distinct trips in each dataset.

E2. **Local and distributed execution.** Compare the local deployment with both distributed configurations using identical trips and request parameters, on the largest dataset of E1 (E3 uses the same data). Include a zone-specific lookup, an origin–destination analysis and a comparison spanning many zones or time windows. Report repeated measurements with central tendency and variability, state the cache and warm-up conditions, and explain the work performed on each instance.

E3. **Contrasting workloads.** Build one workload dominated by requests about a few heavily used zones and a second covering many zones and time periods. **Before running the measurements**, write down which configuration should be most efficient for each workload and why, and deposit this prediction on the course platform (or send it to the instructor). Compare response times, load distribution and resource use as the mixture of requests changes. A wrong prediction that is well explained is valued as much as a correct one.

E4. **A problematic distribution (optional).** This experiment is optional and counts as a bonus. Find a real zone, time window or flow that causes substantial load imbalance, unexpected latency or a misleading analytical result. Investigate its cause, change one design decision and measure both the improvement and its cost elsewhere, with instance-level evidence.

Document the hardware and software versions, resource limits, data sizes, inputs and measurement procedure. Separate the effects of data placement from those of reduced datasets or caching. Several

processes running on one workstation still share CPU, memory and disk; account for this in your interpretation, and do not assume that adding instances improves performance.

Independent verification. Verify at least **two non-trivial results** (one from F2 or F5 and one from F3 or F4) with a separate script that reads the original Parquet files directly, without using your database.

6. Submission and defence

Submission package

By **Friday, 6 November 2026**, submit: the written report (at most 10 pages, excluding appendices) presenting the platform, data preparation, architecture, experiments, verified results and limitations; the complete source code and deployment configuration; the raw measurements; and a README with setup and execution instructions allowing the main experiments to be reproduced.

Defence

Each group gives a **15-minute presentation**, including a live demonstration of at most five minutes, followed by a **10-minute question-and-answer session**. During the questions, the group will face a **live challenge** prepared from its registered choices, for example:

- running services on zones, periods and comparison parameters chosen by the jury;
- importing, through F6, a genuine TLC extract supplied at the defence, then showing that the results change accordingly;
- tracing a reported flow or anomaly back to its source records;
- explaining a file or function of the source code chosen by the jury, by a group member also chosen by the jury.